

Exercises for Machine Learning

Exercises 1

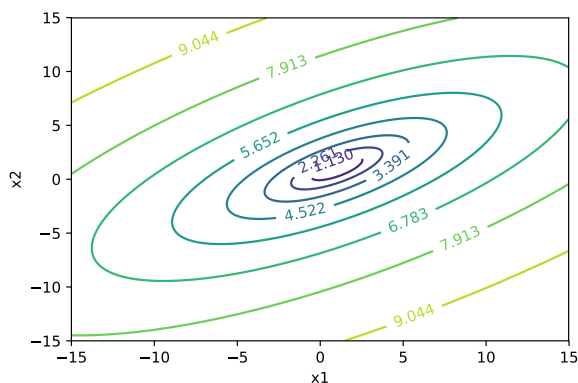
Thomas Dyhre Nielsen

Exercise 1

Consider the function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ defined as

$$f(x_1, x_2) = \ln(x_1^2 + 2 \cdot x_2^2 - 2 \cdot x_1 \cdot x_2 - 2 \cdot x_2 + 2)$$

having the contour lines illustrated here:



1. Using a learning rate of 0.5 (somewhat arbitrarily chosen), perform three iterations of gradient descent starting from the point $(-5, -5)$.
2. Using the same starting point and learning rate, perform three iterations with momentum. How does the two solutions compare?

Solution:

The partial derivatives of f can be found using the chain rule:

$$\begin{aligned}\frac{\partial f}{\partial x_1} &= \frac{(2x_1 - 2x_2)}{x_1^2 + 2 \cdot x_2^2 - 2 \cdot x_1 \cdot x_2 - 2 \cdot x_2 + 2} \\ \frac{\partial f}{\partial x_2} &= \frac{(4x_2 - 2x_1 - 2)}{x_1^2 + 2 \cdot x_2^2 - 2 \cdot x_1 \cdot x_2 - 2 \cdot x_2 + 2}\end{aligned}$$

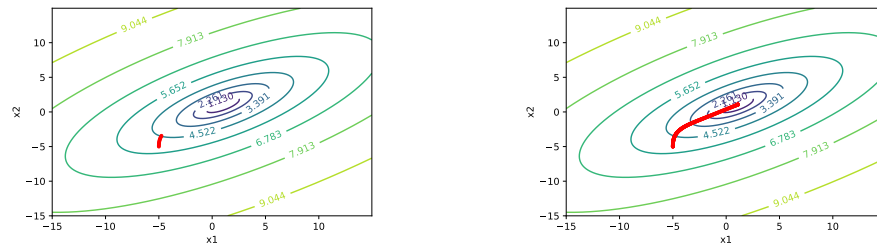
The first three iterations using vanilla gradient descent gives

$(-5.0, -4.837837837837838)$
 $(-4.995380870144192, -4.676168292884549)$
 $(-4.98580088591213, -4.515399090968667)$

and by adding a momentum we get:

$(-5.0, -4.837837837837838)$
 $(-4.995380870144192, -4.530222346938603)$
 $(-4.976595903322989, -4.094088324461926)$

For illustration, the results of the first 10 iterations are shown here:



And here is the pytorch code, which can be used for calculating the gradients:

```
1 import torch
2
3 sigmoid = torch.nn.Sigmoid()
4 relu = torch.nn.ReLU()
5
6 lr = 0.5
7
8 x1 = torch.tensor(-5.0, requires_grad=True)
9 x2 = torch.tensor(-5.0, requires_grad=True)
10
11 for i in range(3):
12     f = torch.log(x1 ** 2 + 2 * x2 ** 2 - 2 * x1 * x2 - 2 * x2 + 2)
13
14     # Do backward propagation in the computational graph
15     f.backward()
16
17     # Get the gradients
18     dx1 = x1.grad
19     dx2 = x2.grad
20
```

```

21 | print("Grad x1 = {}".format(dx1))
22 | print("Grad x2 = {}".format(dx2))
23 |
24 | # Perform a gradient step
25 | x1 = torch.tensor(x1 - lr * dx1, requires_grad=True)
26 | x2 = torch.tensor(x2 - lr * dx2, requires_grad=True)
27 |
28 | print(f"New value for x1: {x1.detach().item()}")
29 | print(f"New value for x2: {x2.detach().item()}")

```

Exercise 2

Consider the small feed-forward neural network in Figure 1. Assume that the hidden unit h has a rectified linear unit (ReLU) as activation function and that the output unit is given a logistic function as activation function. The value provide by the network should be interpreted as an estimate of $P(y | \mathbf{x}, \mathbf{w})$.

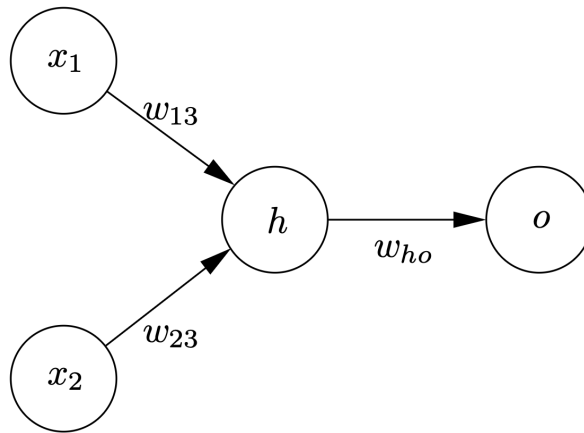


Figure 1: A simple feed forward neural network.

Use the cross-entropy loss:

$$loss(\mathbf{w}) = -t \cdot \ln P(y | \mathbf{x}, \mathbf{w}) - (1 - t) \cdot \ln(1 - P(y | \mathbf{x}, \mathbf{w}))$$

as error function.

1. Specify the computational graph for the model.
2. Derive the back-propagation updating rules for the three weights (w_{13}, w_{23}, w_{ho}) in the network.
3. Specify one training example, and perform one gradient descent updating step based on the chosen example and a suitable learning rate.

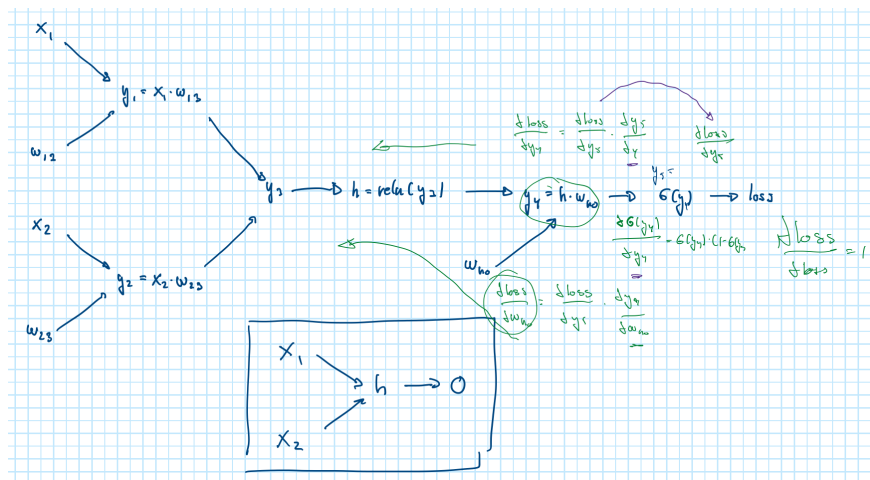
- Implement the model using PyTorch (as in the lecture) and check that your updating rules (gradients) are correct.

Note:

- This exercise is basically an exercise in the systematic use of the chain rule of differentiation (see also the lecture slides). The general rule is given in Section 6.5.2 (Page 206) in the [Deep Learning book](#) and a compressed algorithmic description of back propagation is given in Section 6.5.4.
- Some useful facts about the logistic function can be found in Section 3.10 and 6.2.2.2.

Solution:

Below is a partial solution for the specification of the computational graph.



Python code for calculating gradients:

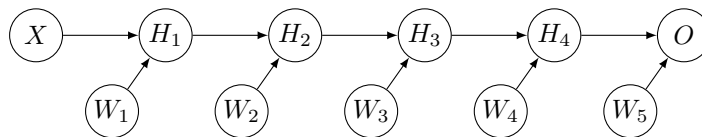
```
1 import torch
2
3 sigmoid = torch.nn.Sigmoid()
4 relu = torch.nn.ReLU()
5
6 x1 = torch.tensor(1.0, requires_grad=True)
7 w13 = torch.tensor(0.5, requires_grad=True)
8
9 x2 = torch.tensor(3.0, requires_grad=True)
10 w23 = torch.tensor(0.2, requires_grad=True)
11
12 t = torch.tensor(1.)
```

```

13 who = torch.tensor(0.1, requires_grad=True)
14
15 y1 = x1*w13
16 y1.register_hook(lambda grad: print("Grad y1 = {}".format(grad)))
17
18 y2 = x2*w23
19 y2.register_hook(lambda grad: print("Grad y2 = {}".format(grad)))
20
21 y3 = y1+y2
22 y3.register_hook(lambda grad: print("Grad y3 = {}".format(grad)))
23
24 h = relu(y3)
25 h.register_hook(lambda grad: print("Grad h = {}".format(grad)))
26
27 y4 = h * who
28 y4.register_hook(lambda grad: print("Grad y4 = {}".format(grad)))
29
30 y5 = sigmoid(y4)
31 y5.register_hook(lambda grad: print("Grad y5 = {}".format(grad)))
32
33 l = t*torch.log(y5) + (1-t)*torch.log(1-y5)
34 l.register_hook(lambda grad: print("Grad l = {}".format(grad)))
35
36 l.backward()
37
38 print("Grad x1 = {}".format(x1.grad))
39 print("Grad x2 = {}".format(x2.grad))
40 print("Grad w13 = {}".format(w13.grad))
41 print("Grad w23 = {}".format(w23.grad))

```

Exercise 3 Implement the following neural network with PyTorch (as in the lecture), where the hidden units and the output unit have a sigmoid activation function:



Let the loss function be $loss = (Y - O)^2$ and assume that we have the training example ($X = 1, O = 1$).

1. Using your implementation, find the partial derivatives for each of the weights in the network.
2. How do the derivatives compare? Can you see a general pattern and what are the implications for gradient descent?

3. Can you argue for why you see this particular pattern? Hint: Consider the local contributions to the gradient from the sigmoid functions (what is the maximum value of the local gradient?).

Solution:

```
1 import torch
2
3 sigmoid = torch.nn.Sigmoid()
4
5 x = torch.tensor(1.0, requires_grad=True)
6 w1 = torch.tensor(1.0, requires_grad=True)
7 w2 = torch.tensor(1.0, requires_grad=True)
8 w3 = torch.tensor(1.0, requires_grad=True)
9 w4 = torch.tensor(1.0, requires_grad=True)
10 w5 = torch.tensor(1.0, requires_grad=True)
11
12 y1 = sigmoid(x*w1)
13 y1.register_hook(lambda grad: print(f"Grad y1 = {grad}"))
14
15 y2 = sigmoid(y1*w2)
16 y2.register_hook(lambda grad: print(f"Grad y2 = {grad}"))
17
18 y3 = sigmoid(y2*w3)
19 y3.register_hook(lambda grad: print(f"Grad y3 = {grad}"))
20
21 y4 = sigmoid(y3*w4)
22 y4.register_hook(lambda grad: print(f"Grad y4 = {grad}"))
23
24 y5 = sigmoid(y4*w5)
25 y5.register_hook(lambda grad: print(f"Grad y5 = {grad}"))
26
27 e = (1.0 - y5)**2
28
29 e.backward()
30
31 print("Grad x = {}".format(x.grad))
32 print("Grad w1 = {}".format(w1.grad))
33 print("Grad w2 = {}".format(w2.grad))
34 print("Grad w3 = {}".format(w3.grad))
35 print("Grad w4 = {}".format(w4.grad))
36 print("Grad w5 = {}".format(w5.grad))
37
38 print("Done")
```

Exercise 4 Experiment with the tensorflow playground

<https://playground.tensorflow.org>

- Use different data sets and neural network structures (varying the number

of layers and neurons). How does the different network structures affect the learning ability? Can you derive any useful insights by considering the features learned at the hidden units.

- What is the effect of varying the learning rate and applying regularization (L_1 and L_2 in combination with a regularization rate)?